



UNIVERSITÉ SIDI MOHAMED BEN ABDELLAH FACULTÉ DES SCIENCES ET TECHNIQUES FES

« Ingénierie Electrique et Systèmes Embarqués »

COMPTE RENDU : Circuits intégrés en technologie CMOS

Préparé par :

FRANKLIN HAMUNYEMBA

AMANSOUR MOHAMMED

BENCHEKROUN LAKRIMI MOHAMMED

Supervisé par :

Pr. Kamal ZARED

Pr. Hicham AKHAMAL

Introduction Générale

Dans le cadre du module "Ingénierie des Circuits Intégrés", ce travail pratique vise à maîtriser la chaîne de conception d'un circuit intégré numérique. L'outil utilisé est **Cadence Virtuoso**, couplé au kit technologique **gpdk180nm**.

L'objectif principal de ce travail est de concevoir et d'analyser des circuits microélectroniques en technologie CMOS.

Plus précisément, ce projet comporte **deux réalisations essentielles** :

1. **La conception complète d'un inverseur CMOS**, incluant la création du schéma, du symbole, du testbench, les analyses DC et transitoires, puis la génération et la vérification du layout (DRC/LVS).
2. **La conception d'une porte logique NAND CMOS**, suivant une méthodologie similaire : schéma, symbole, testbench, simulation et implémentation du layout.

Ces deux montages permettent d'illustrer de manière pratique l'ensemble du flux de conception full-custom dans Cadence, depuis la modélisation fonctionnelle jusqu'à la représentation physique du circuit.

Table des matières

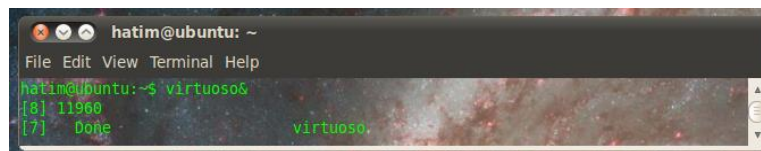
Mise en place de l'environnement de travail	4
Manipulation 1 : L'Inverseur CMOS	4
But de la manipulation :	4
Déroulement de la Conception et Simulations :	4
1. Création de la bibliothèque et Choix technologique	4
2. Conception du schéma de l'inverseur	5
3. Création du symbole	6
4. Création du Test Bench	8
5. Analyse du circuit en mode DC	10
6. Simulation transitoire	12
7. Réalisation du Layout et Vérification	13
8. Conclusion	15
Manipulation 2 : La Porte Logique NAND	16
But de la manipulation :	16
Déroulement de la Conception et Simulations :	16
1. Création de la bibliothèque et Choix technologique	16
2. Création du symbole	17
3. Création du Test Bench	18
4. Simulation Transitoire et Analyse des Résultats	19
5. Réalisation du Layout et Vérification	20
6. Conclusion	23

Mise en place de l'environnement de travail

Avant d'entamer les manipulations, nous avons configuré l'environnement logiciel **Cadence Virtuoso** et préparé la bibliothèque de travail.

Nous avons ouvert un terminal et exécuté la commande *virtuoso &* pour lancer la plateforme. L'environnement comprend trois modules principaux que nous avons exploités:

- **Virtuoso Schematic Editor** : Pour la saisie du schéma.
- **ADE L (Analog Design Environment)** : Pour les simulations.
- **Layout XL** : Pour le dessin des masques.



Manipulation 1 : L'Inverseur CMOS

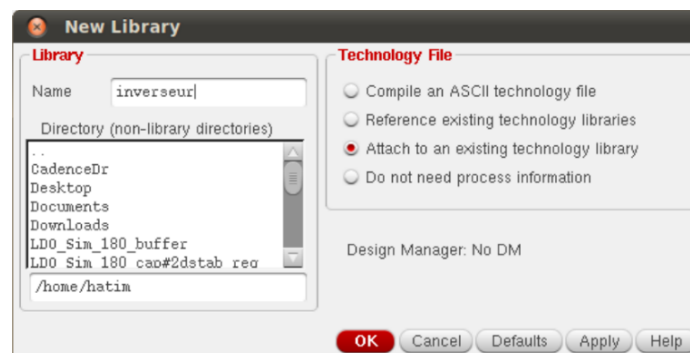
But de la manipulation :

Le but est de comprendre le comportement électrique d'un inverseur CMOS, d'analyser ses caractéristiques de transfert (DC) et temporelles (Transitoire), et de générer son layout physique pour valider la fabrication.

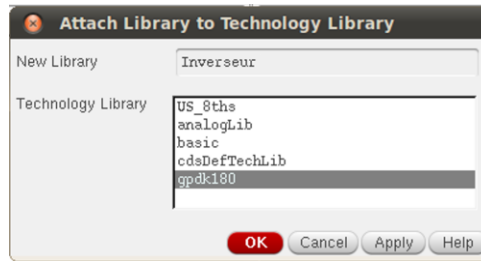
Déroulement de la Conception et Simulations :

1. Création de la bibliothèque et Choix technologique

Avant de commencer le schéma, nous avons ouvert le **Library Manager** pour créer notre espace de travail. Nous avons créé une nouvelle bibliothèque nommée inverseur :



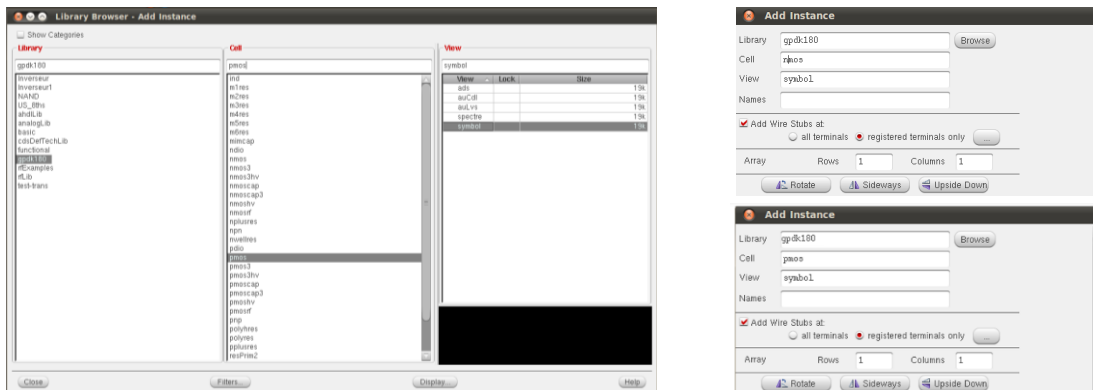
Nous avons attaché cette bibliothèque à la technologie **gpd180** (Generic Process Design Kit 180nm) :



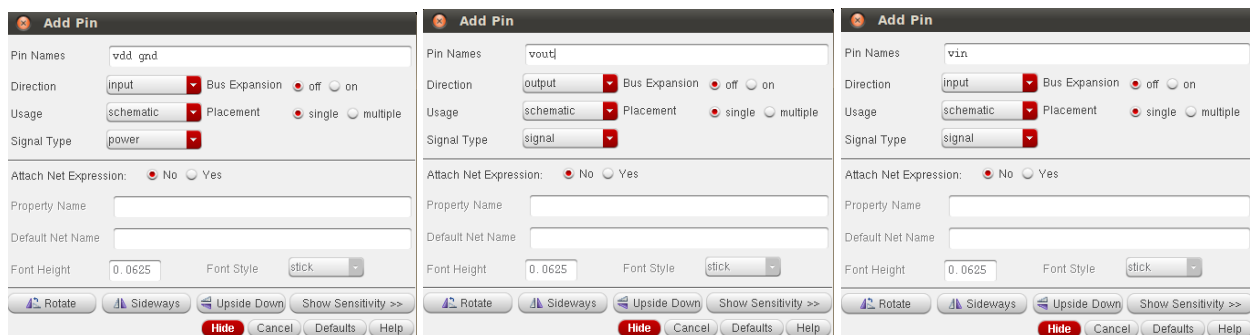
Cette étape est primordiale car elle importe les règles de dessin et les modèles de transistors nécessaires à la simulation et au layout.

2. Conception du schéma de l'inverseur

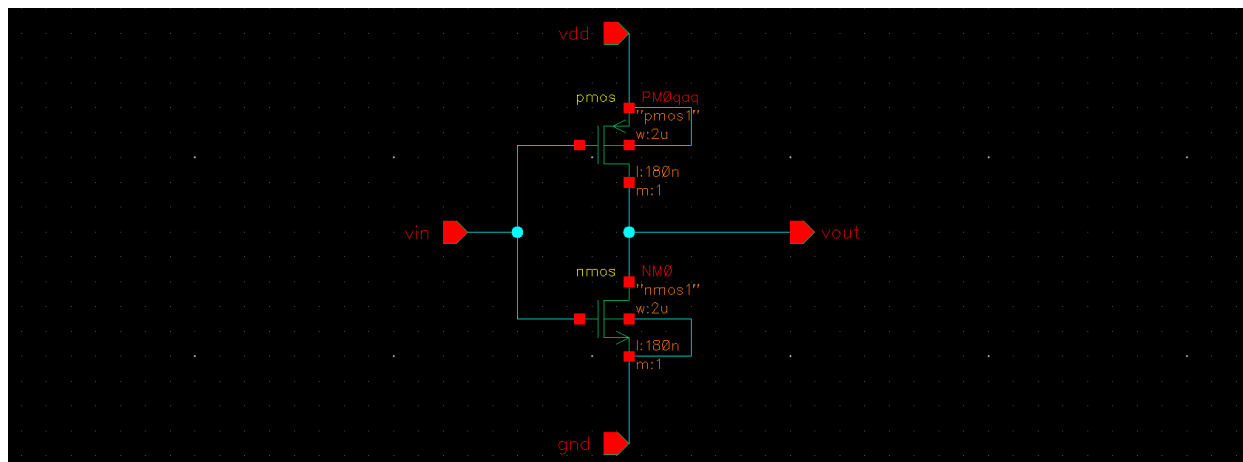
Nous avons d'abord créé une **vue schématique** puis câblé le circuit en utilisant les composants du **kit gpd180**. À partir de cette bibliothèque, nous avons inséré un **transistor PMOS** et un **transistor NMOS** dans l'éditeur de schéma, en cliquant sur **"I"**.



Nous avons utilisé le bouton **"P"** afin d'ouvrir la fenêtre d'insertion des pins. À partir de cette fenêtre, nous avons ajouté : **VDD**, **GND**, V_{in} et V_{out} .



Le **transistor PMOS** a été connecté à V_{DD} , tandis que le **transistor NMOS** a été relié à la masse **GND**. Ensuite, nous avons relié les **grilles** des deux transistors pour former l'entrée V_{in} , et leurs **drains** ont été connectés ensemble afin d'obtenir la sortie V_{out} . De plus, nous avons connecté le **substrat du PMOS** à V_{DD} , et le **substrat du NMOS** à **GND**, afin d'assurer le fonctionnement correct des transistors et d'éviter tout effet parasite.



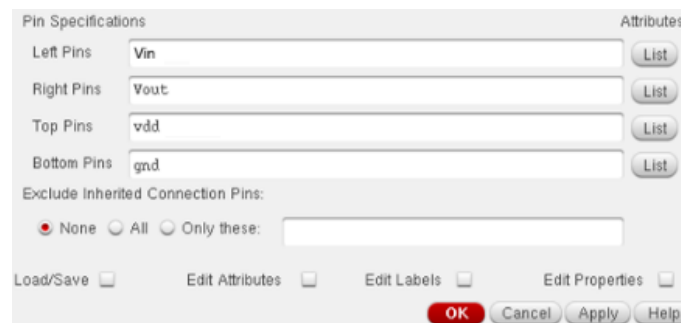
3. Création du symbole

Pour faciliter la réutilisation de notre inverseur dans d'autres circuits, nous avons créé son symbole graphique en suivant trois étapes :

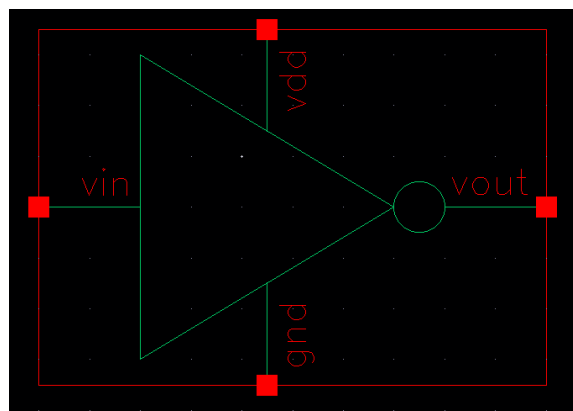
Depuis la fenêtre du schéma, nous avons accédé au menu Create → CellView → From CellView en conservant le nom proposé par défaut.



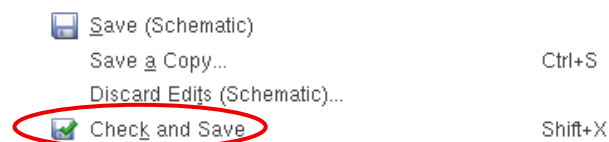
Configuration des pins : Dans la boîte de dialogue, nous avons déplacé les pins vers les champs correspondants pour définir leur position.



Édition Graphique : L'outil a d'abord généré un symbole rectangulaire par défaut. Nous l'avons ensuite modifié manuellement afin d'obtenir la forme géométrique normalisée d'un inverseur. Les pins d'entrée, de sortie et d'alimentation (V_{in} , V_{out} , VDD, GND) ont été repositionnés de manière cohérente afin d'assurer une utilisation intuitive du composant dans les testbenches, comme illustré ci-dessous.



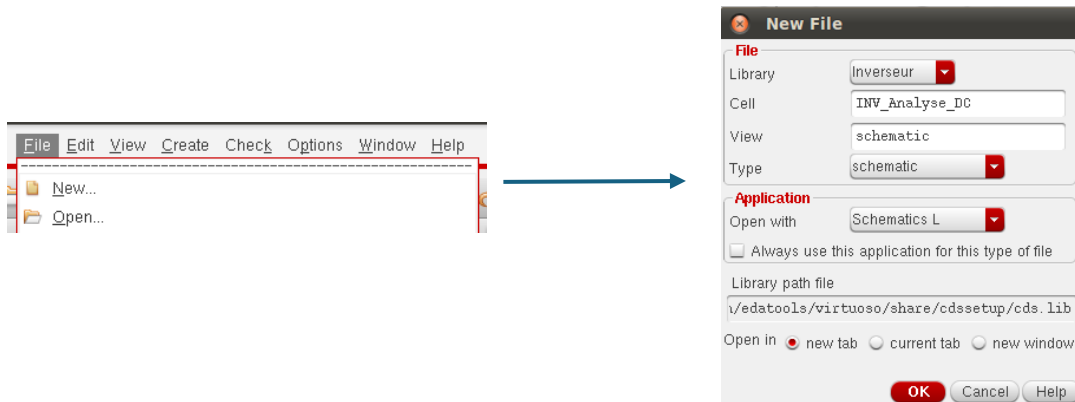
Une **vérification** finale (*Check and Save*) assure la conformité du symbole avant son utilisation dans les étapes suivantes.



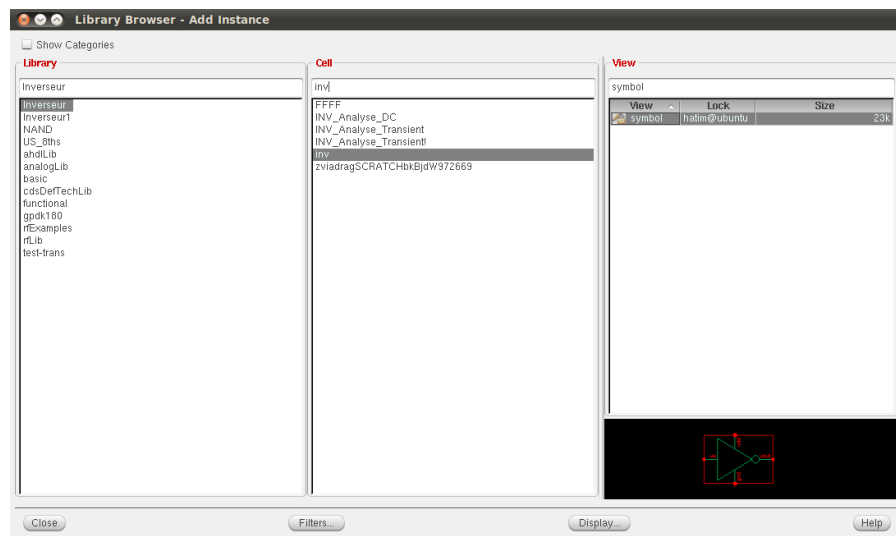
4. Création du Test Bench

Pour valider le fonctionnement de l'inverseur, nous avons élaboré un banc de test (Test Bench) en suivant ces étapes :

Dans notre bibliothèque de travail, nous avons créé une nouvelle cellule schéma nommée INV_Analyse_DC via le menu File -> New -> Cell View.

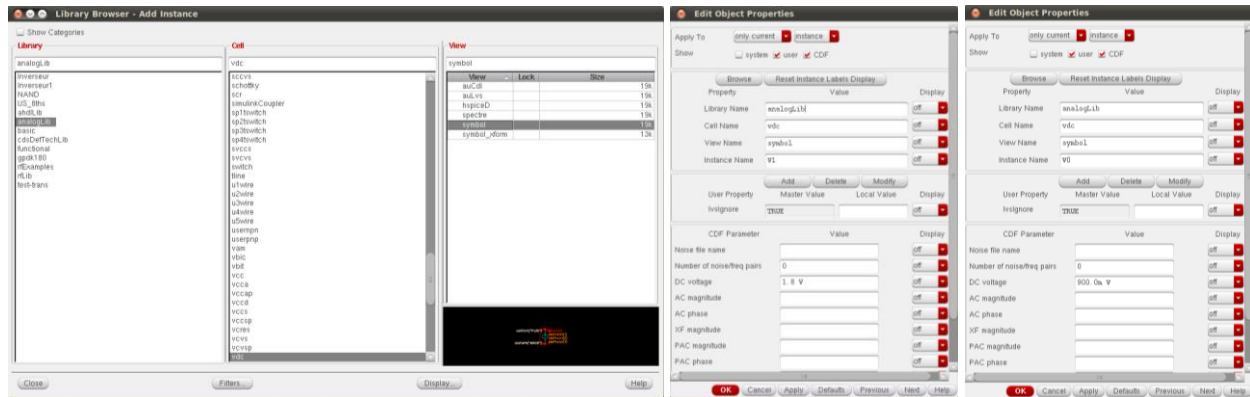


Nous avons instancié le symbole de l'inverseur créé précédemment en utilisant la touche “I” et en le sélectionnant dans le répertoire Inverseur.

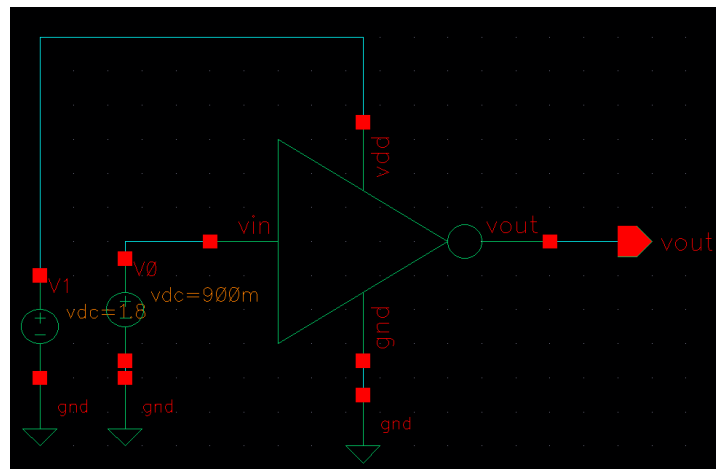


Nous avons complété le circuit en ajoutant les sources de tension nécessaires. Pour cela, nous avons utilisé la bibliothèque **analogLib** et sélectionné l'élément **vdc** afin de créer les deux générateurs :

- Une tension d'alimentation **Vdd = 1.8 V**,
- Une tension d'entrée continue **Vin = 0.9 V**.



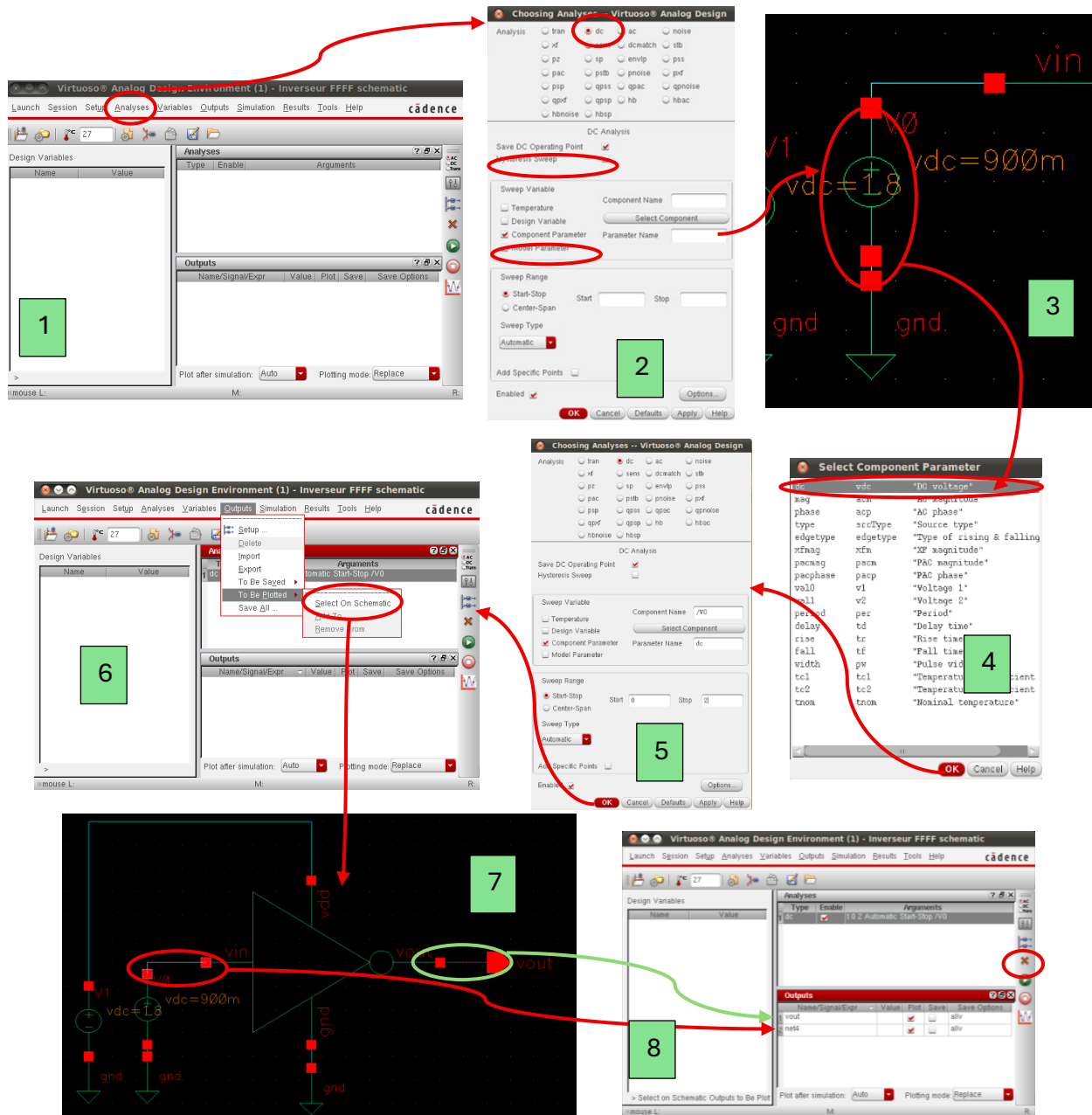
Une fois l'assemblage de ces éléments terminé, nous avons obtenu le schéma complet du banc de test, prêt pour la simulation, comme illustré ci-dessous :



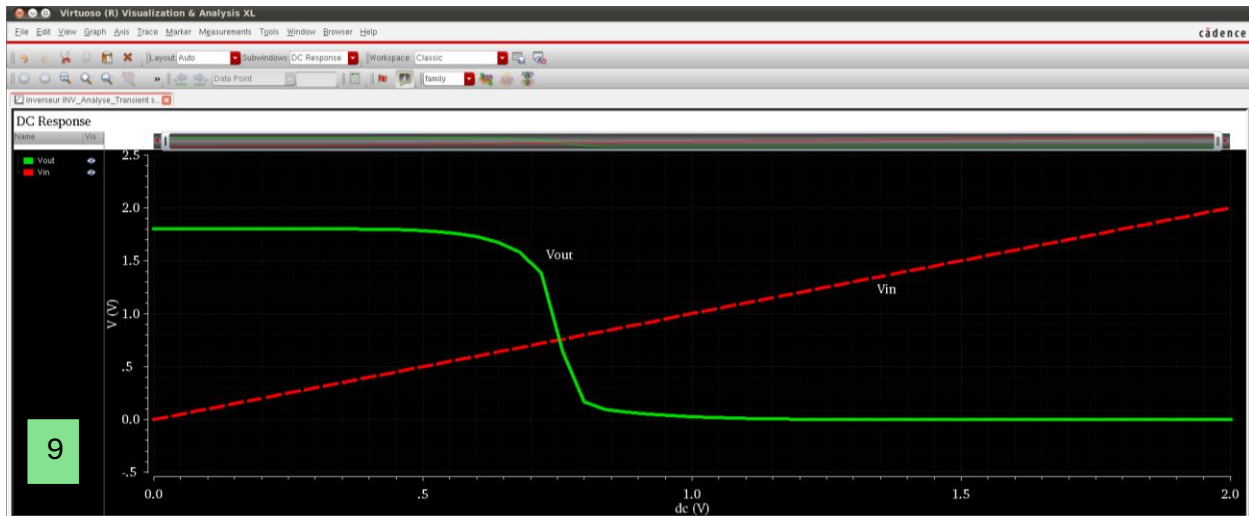
5. Analyse du circuit en mode DC

Pour analyser la caractéristique de transfert statique de notre inverseur, nous avons utilisé l'outil **ADE L**.

Depuis l'éditeur de schéma, nous avons ouvert l'outil via Launch → ADE L. En suivant les étapes illustrées par les captures d'écran jointes, nous avons configuré une analyse DC en définissant la tension d'entrée comme variable à balayer.



Une fois la simulation lancée (Simulation → Run), nous avons obtenu la caractéristique de transfert suivante :



Le graphe confirme le comportement inverseur attendu :

- Pour V_{in} faible (proche de $0V$), V_{out} est à son maximum ($1.8V$), car le transistor PMOS est passant et le NMOS bloqué.
- Pour V_{in} élevé (proche de $1.8V$), V_{out} chute à $0V$, car le NMOS devient passant et le PMOS se bloque.

La zone de transition abrupte autour de $V_{in} \approx 0.8V - 0.9V$ démontre un bon gain en tension et un seuil de basculement proche de $V_{dd}/2$, idéal pour l'immunité au bruit.

Nous avons alors confirmé avec succès le bon fonctionnement du circuit.

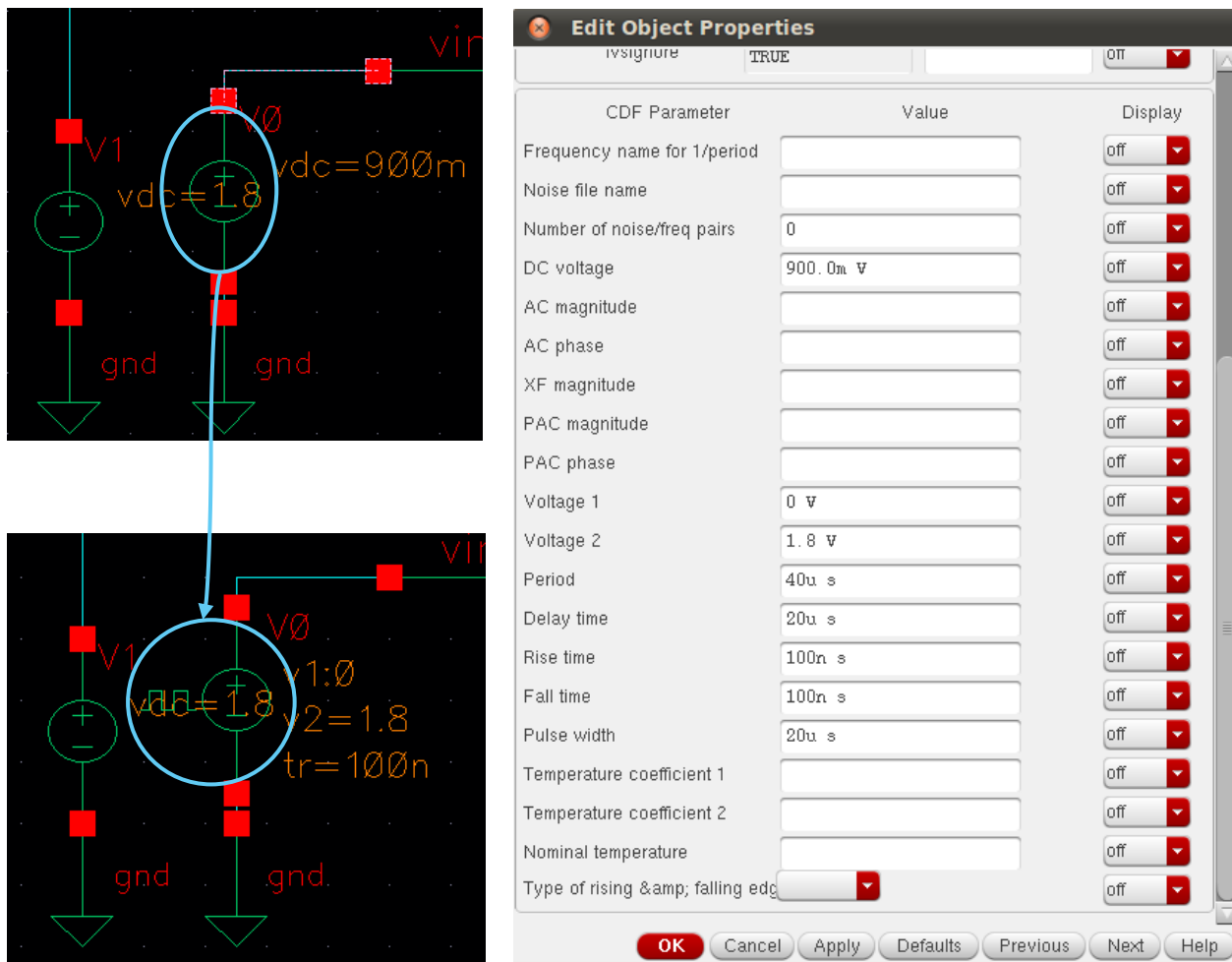
Pour documenter notre travail, nous avons exporté les courbes obtenues. Depuis la fenêtre *Virtuoso Visualization & Analysis XL*, nous avons utilisé le menu File → Save Image afin d'enregistrer les graphiques de simulation.



6. Simulation transitoire

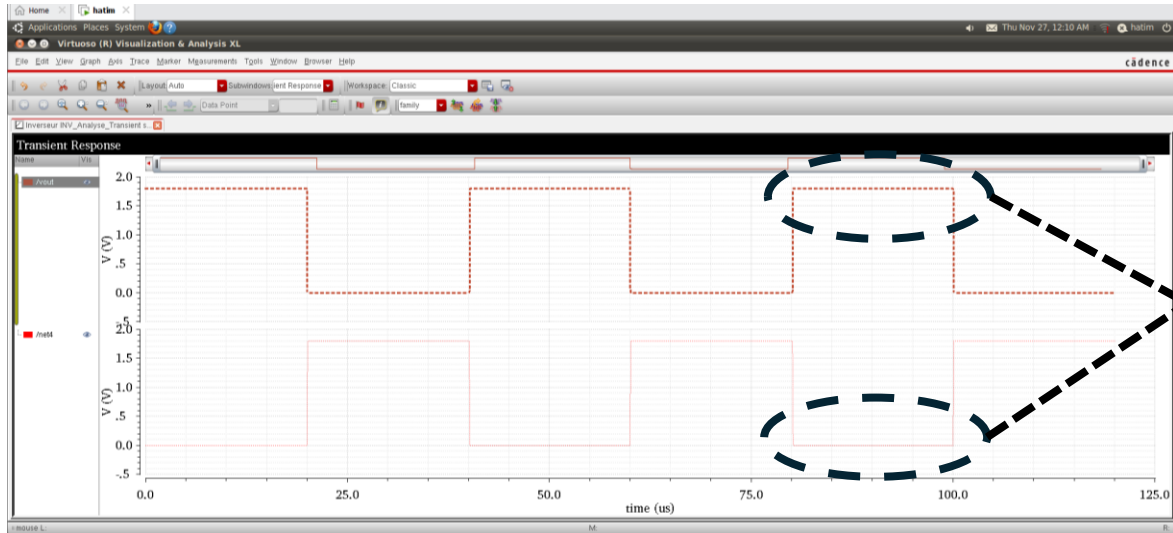
Afin d'analyser le comportement temporel de l'inverseur (temps de montée, temps de descente et propagation), nous devons passer d'un régime statique (DC) à un régime dynamique.

Pour ce faire, nous avons modifié notre banc de test en remplaçant le générateur de tension continue (V_{dc}) à l'entrée par un générateur d'impulsions (V_{pulse}), sélectionné dans la même bibliothèque *analogLib*. Ce changement est nécessaire pour appliquer un signal carré périodique et observer le basculement de la porte.



Pour cette analyse, nous avons configuré les paramètres du générateur d'impulsions (V_{pulse}) afin d'obtenir un signal carré bien défini. Nous avons fixé les niveaux de tension à **0V** et **1.8V** pour correspondre à la logique du circuit, avec une **période de 40μs** et une largeur d'impulsion de **20μs**. Enfin, dans l'environnement de simulation (ADE L), nous avons réglé le temps d'arrêt (*Stop Time*) à **125μs** pour pouvoir observer une période complète du signal.

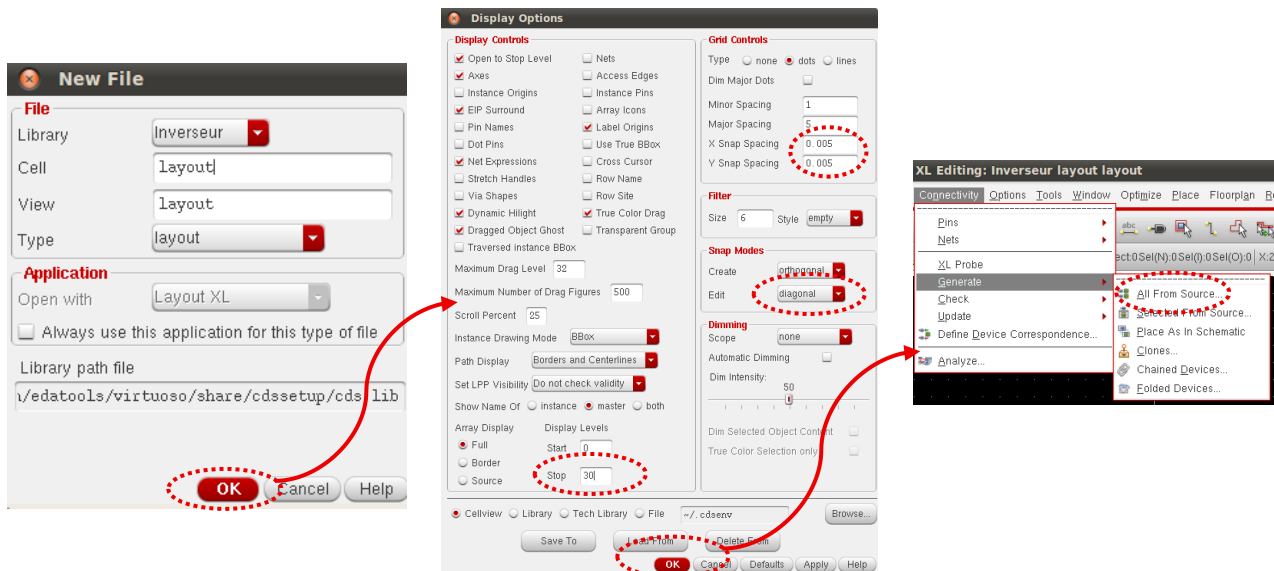
La simulation transitoire met en évidence le comportement dynamique de l'inverseur CMOS:

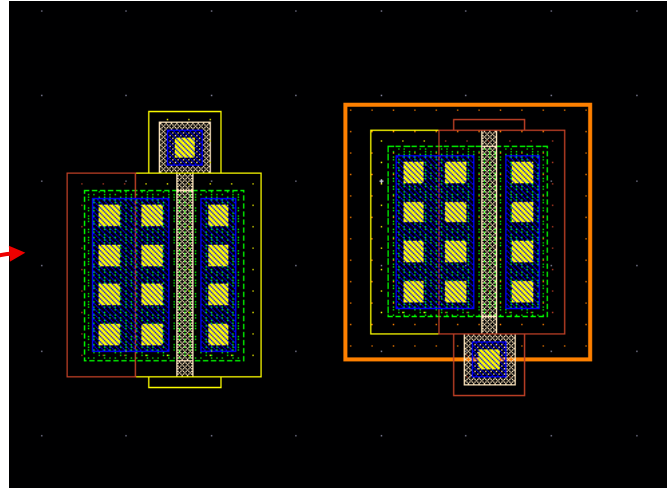
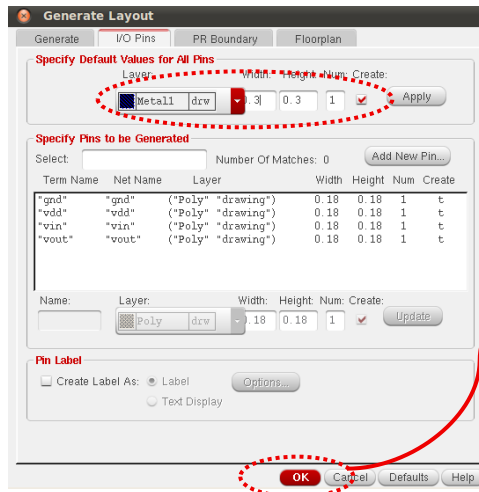


Lorsque le signal d'entrée à forme impulsionnelle commute entre les niveaux logiques haut et bas, la sortie bascule de manière complémentaire avec des temps de montée et de descente cohérents, confirmant la capacité du circuit à suivre correctement les variations rapides de l'entrée.

7. Réalisation du Layout et Vérification

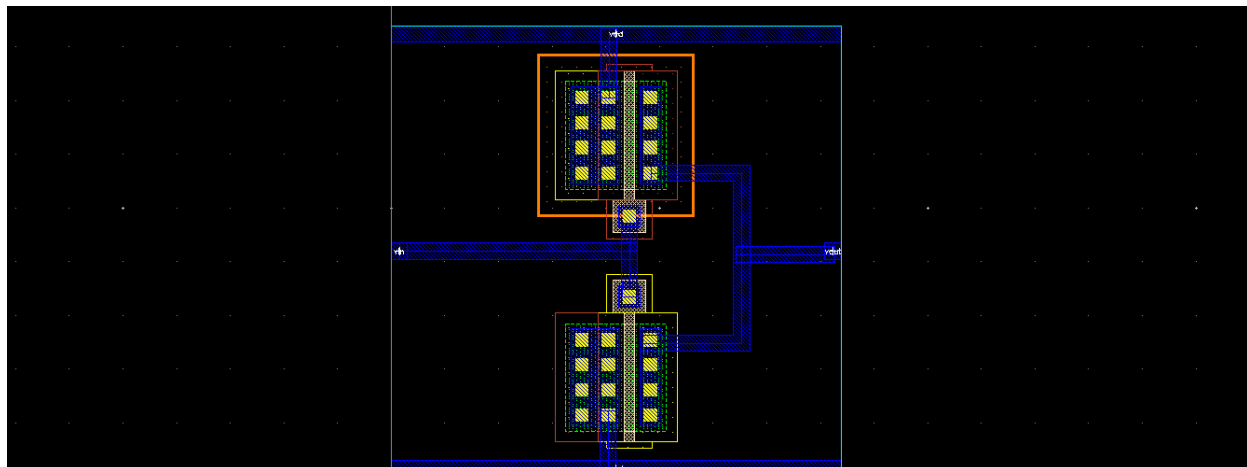
En suivant les étapes illustrées par les captures d'écran jointes, nous avons utilisé **Layout XL** pour générer les masques physiques à partir du schéma précédent.





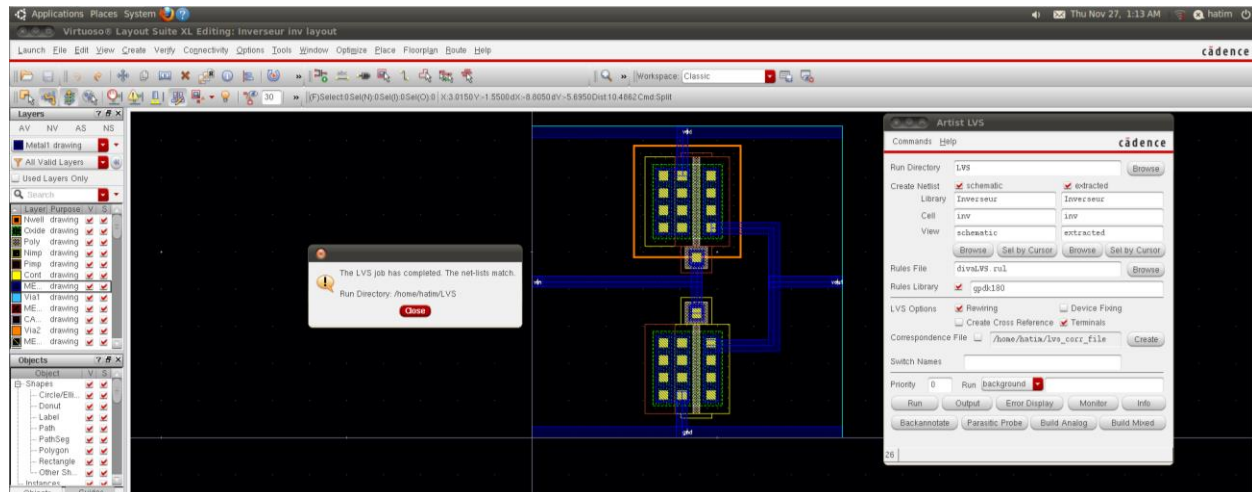
Après avoir généré les composants à partir du schéma, nous avons obtenu les empreintes des transistors placées mais non connectées. Nous avons alors procédé au routage manuel pour finaliser le circuit physique.

En utilisant la couche **Métal 1** (visible en bleu sur l'interface), nous avons tracé les pistes pour relier les différentes bornes. Une fois ce câblage terminé, nous avons obtenu le layout final illustré ci-dessous :

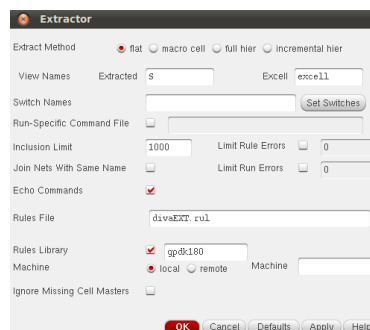


Une fois le routage terminé, il était impératif de valider notre dessin vis-à-vis des contraintes technologiques. Chaque manipulation de layout doit être rigoureusement vérifiée pour garantir le respect des règles de fabrication du kit gpd180. Pour ce faire, nous avons lancé l'outil de vérification via le menu Verify → DRC.

Afin d'identifier et de comprendre les éventuelles violations, nous avons utilisé la fonction Verify → Markers → Explain, qui permet d'afficher une description détaillée des erreurs directement sur le layout.



Les vérifications DRC et LVS a confirmé que le layout correspond exactement au schéma électrique, en validant que toutes les connexions, tailles de transistors et configurations internes sont identiques entre les deux représentations du circuit.



Enfin, nous avons procédé à l'extraction du layout via le menu **Verify → Extract**. Cette opération a généré la vue « extracted », nécessaire pour les simulations post-layout.

8. Conclusion partielle

Cette première manipulation nous a permis de maîtriser le flux de conception complet d'un inverseur CMOS sous Cadence Virtuoso. Les simulations analogiques ont validé les caractéristiques statiques (DC) et dynamiques (transitoire) du circuit, confirmant sa fonction d'inversion. Enfin, la réalisation du layout et sa validation par les tests DRC et LVS ont garanti la conformité physique du dessin vis-à-vis des règles technologiques et du schéma électrique.

Manipulation 2 : La Porte Logique NAND

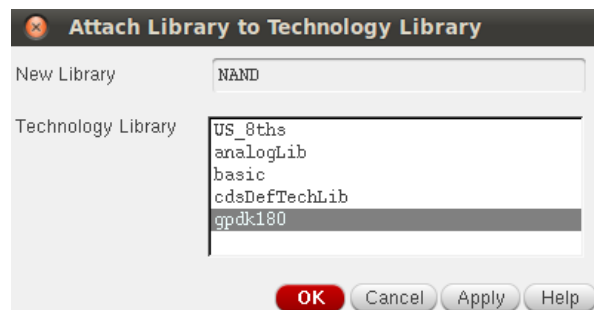
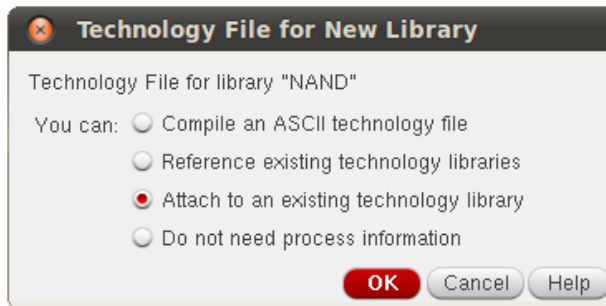
But de la manipulation :

L'objectif de cette seconde manipulation est d'appliquer le flux de conception "Full-Custom" à un circuit logique combinatoire plus complexe : une porte **NAND** à deux entrées. Il s'agit de modéliser le schéma transistor, de créer son symbole, de vérifier sa table de vérité par simulation transitoire, et enfin de réaliser son layout physique.

Déroulement de la Conception et Simulations :

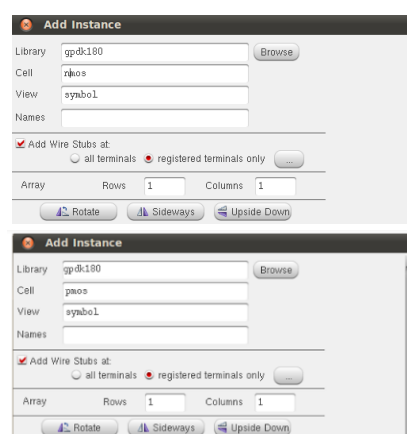
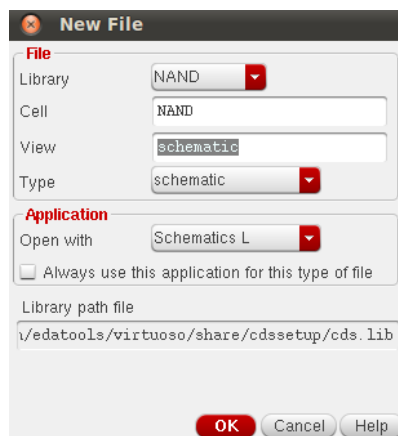
1. Création de la bibliothèque et Choix technologique

Nous avons commencé par créer une nouvelle bibliothèque nommée **NAND** attachée au kit technologique **gpd180**.



Ensuite, nous avons créé une nouvelle vue schématique pour dessiner la structure interne de la porte NAND CMOS, conformément à la topologie théorique :

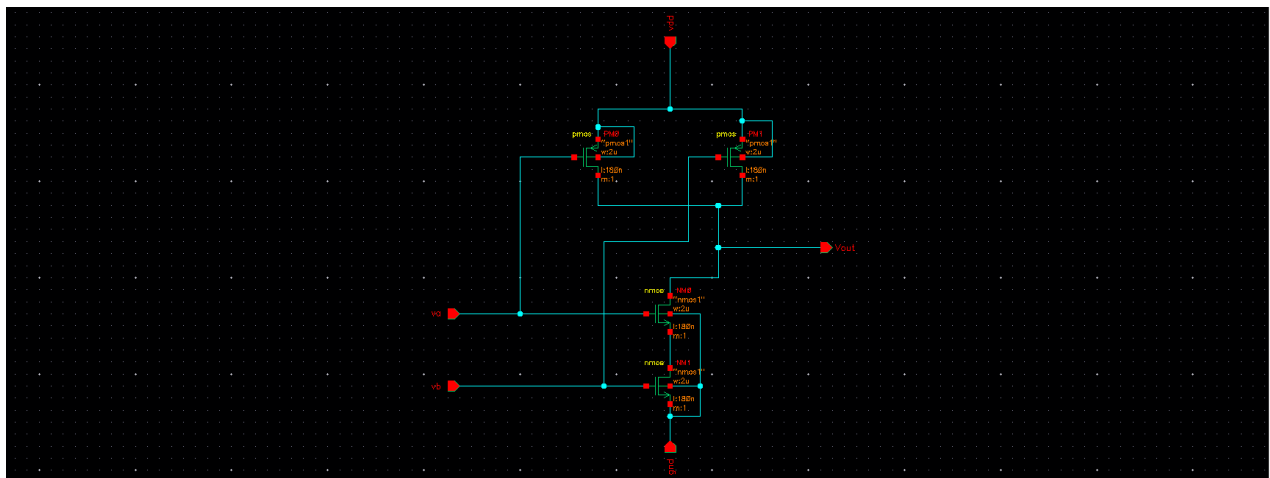
- **Réseau de Pull-Up (PU)** : Deux transistors **PMOS** montés en **parallèle** entre l'alimentation V_{DD} et la sortie V_{out} .
- **Réseau de Pull-Down (PD)** : Deux transistors **NMOS** montés en **série** entre la sortie V_{out} et la masse **GND**.



Nous avons utilisé les dimensions standards pour cette technologie ($L = 180\text{nm}$, $W = 2\mu\text{m}$). Pour établir les connexions externes, nous avons activé la commande de création de pins (raccourci clavier P). Dans la fenêtre de configuration, nous avons spécifié les noms des terminaux et sélectionné leurs directions respectives comme suit :

- **Entrées** : v_a et v_b (Direction : Input).
- **Sortie** : v_{out} (Direction : Output).
- **Alimentation** : v_{dd} et gnd (Direction : Input/Output).

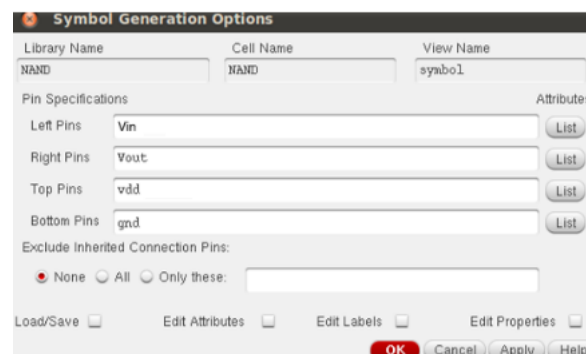
Une vérification finale (**Check and Save**) nous a permis de valider l'absence d'erreurs de connexion ou de nœuds flottants.



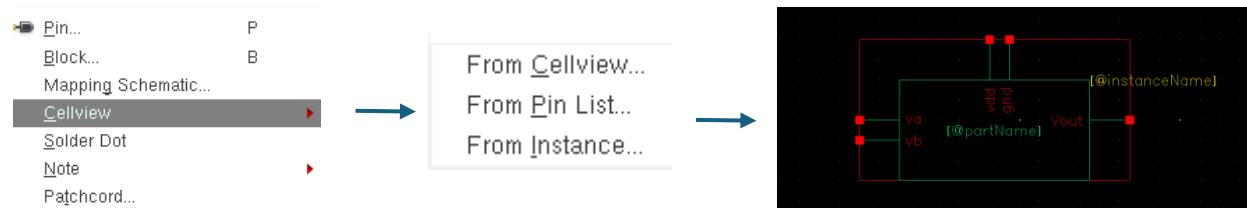
2. Création du symbole

Pour pouvoir utiliser cette porte logique dans des simulations ultérieures, nous avons généré son symbole via le menu *Create* → *CellView* → *From CellView*

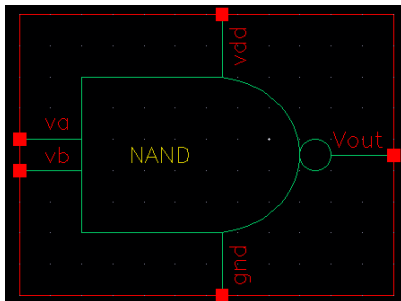
Dans la boîte de dialogue, nous avons déplacé les pins vers les champs correspondants pour définir leur position.



L'outil a généré un rectangle par défaut.



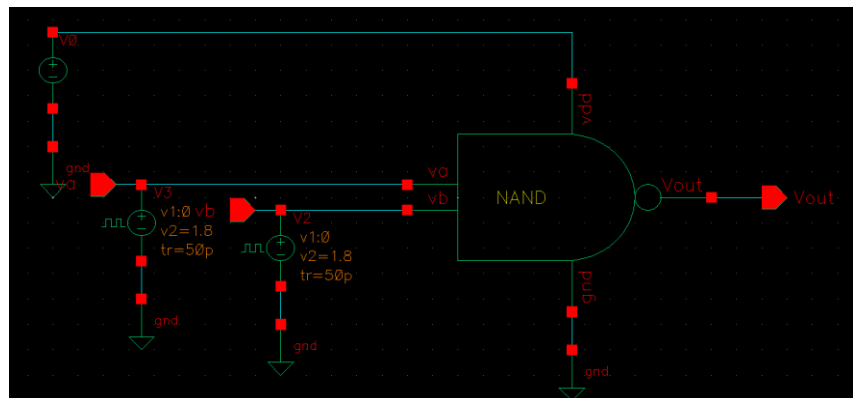
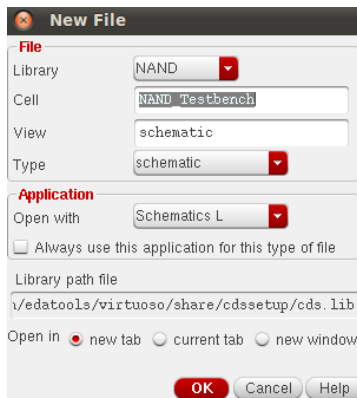
Nous avons supprimé ce cadre pour dessiner manuellement la forme normalisée d'une porte **NAND** en utilisant les outils de dessin géométrique.



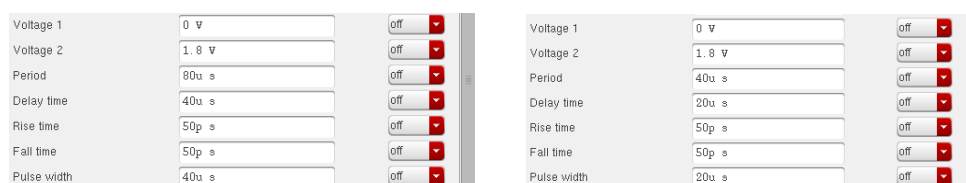
- La sortie v_{out} à droite.
- Les entrées v_a et v_b à gauche.
- Les alimentations v_{dd} et gnd en haut et en bas.

3. Création du Test Bench

Pour valider le fonctionnement logique, nous avons créé une nouvelle cellule nommée **NAND_Testbench**. Nous avons instancié notre symbole **NAND** et connecté les sources nécessaires provenant de la bibliothèque **analogLib** :

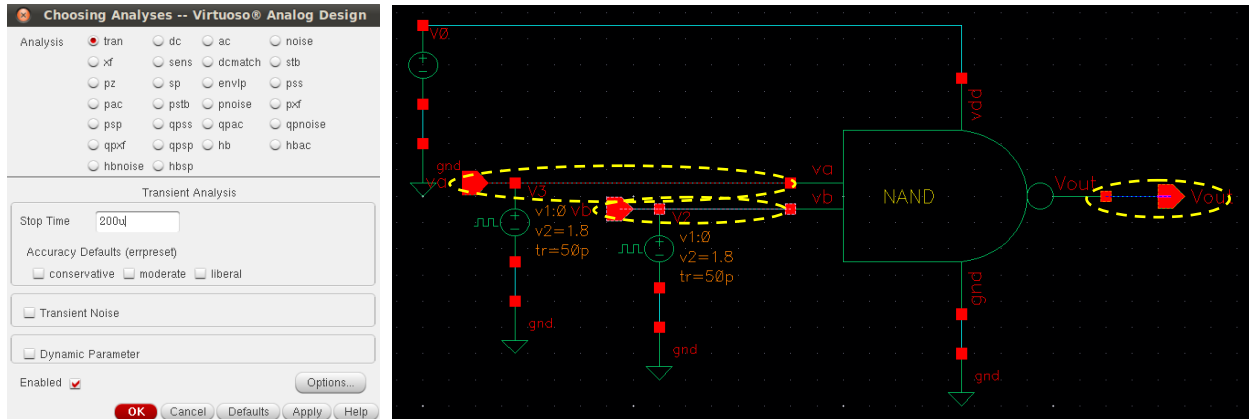


Deux sources impulsionnelles V_{pulse} pour les entrées v_a et v_b , configurées avec des fréquences différentes afin de balayer toutes les combinaisons logiques possibles.

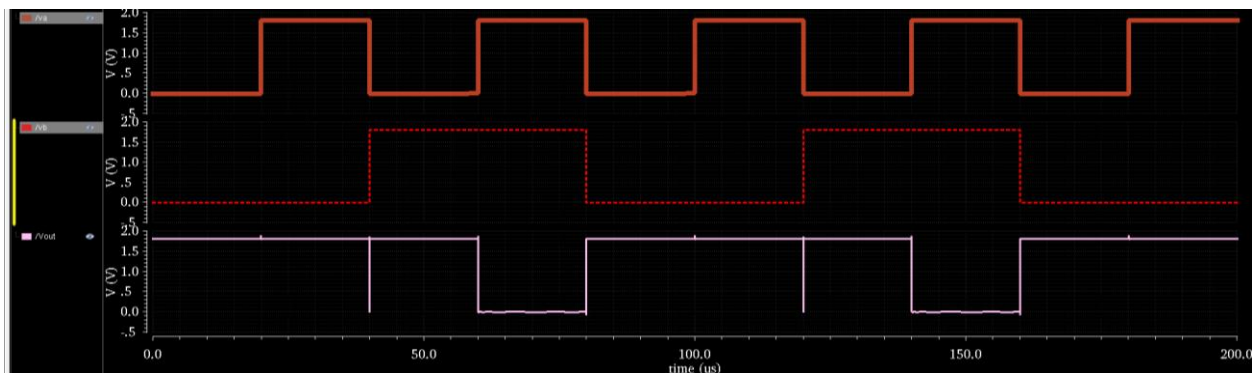


4. Simulation Transitoire et Analyse des Résultats

Nous avons configuré l'environnement de simulation **ADE L** pour effectuer une analyse transitoire (**tran**).



- **Temps de simulation** : Nous avons choisi une durée suffisante (200ns) pour observer au moins deux périodes complètes des signaux d'entrée.
- **Signaux à tracer** : Nous avons sélectionné les entrées v_a , v_b et la sortie V_{out} depuis le schéma.



Après avoir lancé la simulation, nous avons obtenu les chronogrammes montrant l'évolution des tensions. L'analyse des courbes confirme le comportement d'une porte NAND :

- La sortie V_{out} est à l'état bas (0V) **uniquement** lorsque les deux entrées v_a et v_b sont simultanément à l'état haut (1.8V).
- Dans tous les autres cas, la sortie reste à l'état haut (1.8V).

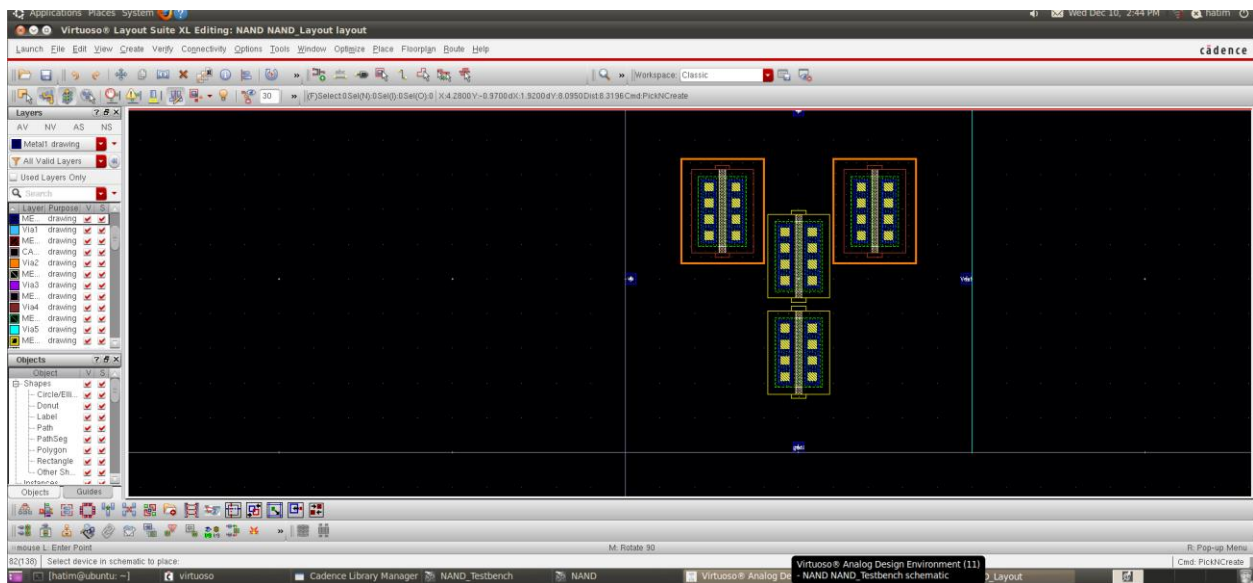
A	B	Q
0	0	1
0	1	1
1	0	1
1	1	0

Ce résultat valide la table de vérité de la porte NAND et le bon dimensionnement des transistors.

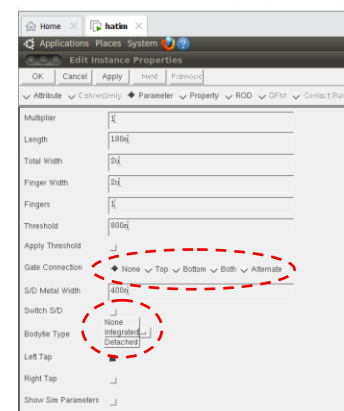
5. Réalisation du Layout et Vérification

La dernière étape consistait à réaliser le dessin des masques de la porte NAND. Depuis l'éditeur de schéma, nous avons lancé **Layout XL** et importé les composants via la commande *Connectivity* → *Generate* → *All From Source*.

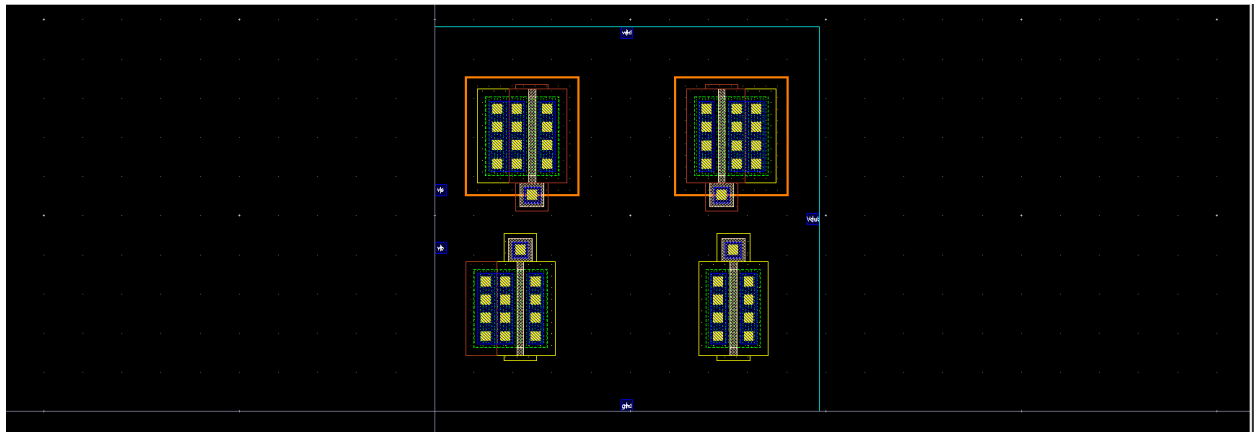
Lors de la génération initiale du layout à partir du schéma, les transistors apparaissent sous leur forme standard, ne montrant que les zones de diffusion (**Drain/Source**) ainsi que le polysilicium de la grille. Pour assurer une polarisation correcte, il est nécessaire d'intégrer les prises de **substrat** ainsi que les contacts de **grille**.



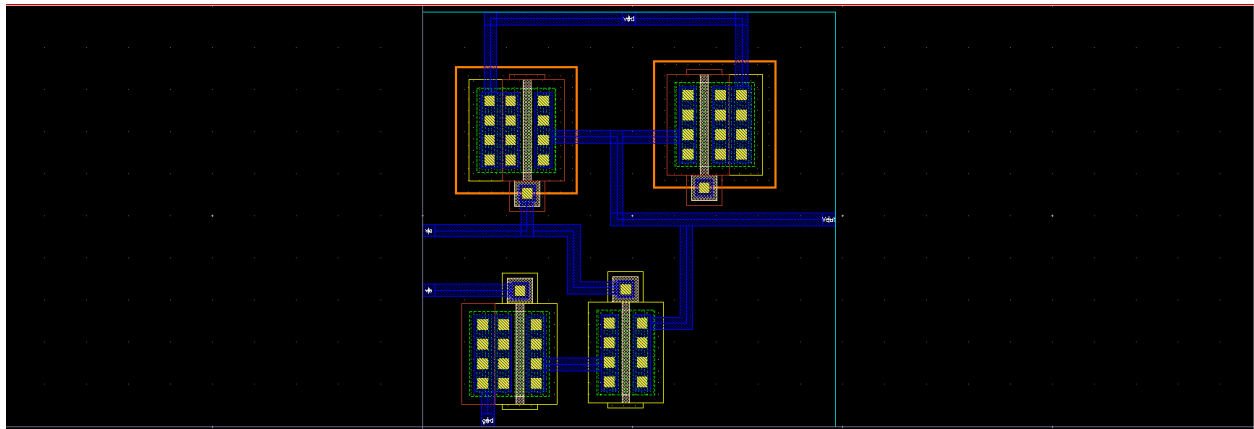
Nous avons sélectionné chaque transistor dans l'éditeur de layout. Après, nous avons ouvert la fenêtre de propriétés (raccourci **Q**). Et finalement, dans les paramètres de l'instance, nous avons activé l'option **Bodytie Type**. Aussi, nous avons sélectionné "Top" ou "Bottom" (selon l'orientation du routage) pour faire apparaître automatiquement le contact sur la grille.



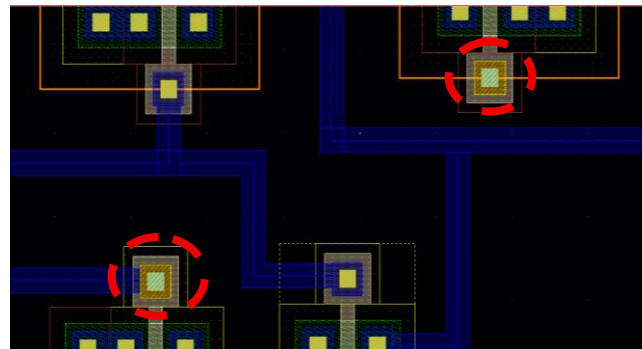
Après avoir cliqué sur **Apply** puis **OK**, nous avons observé la mise à jour immédiate des composants sur le layout, comme le montre la figure ci-dessous :



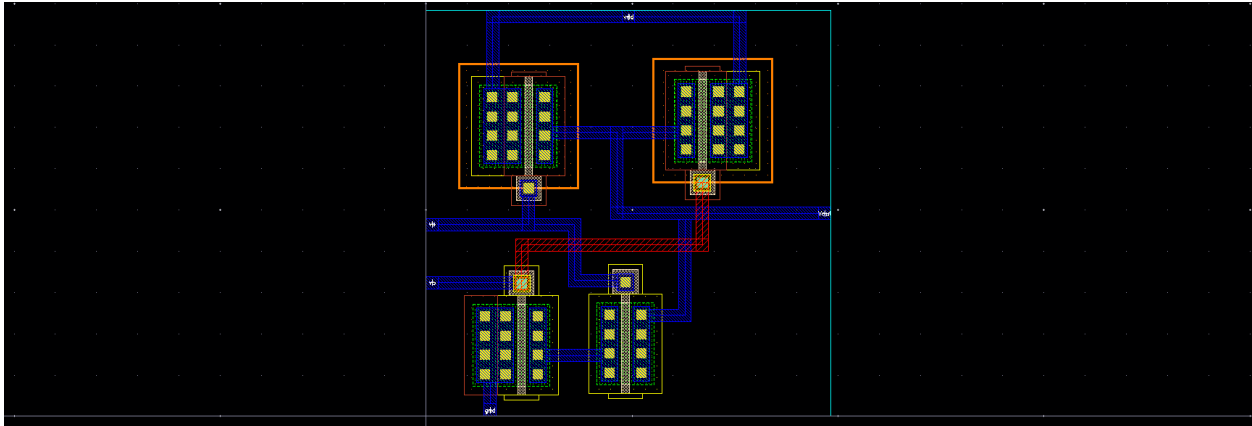
En utilisant la couche **Métal 1** (visible en bleu sur l'interface), nous avons tracé les pistes pour relier les différentes bornes. Une fois ce câblage terminé, nous avons obtenu le layout illustré ci-dessous :



Par la suite, nous avons constaté qu'il était impossible de relier directement les grilles des transistors **PM1** et **NM1** en utilisant uniquement la couche Métal 1, en raison des croisements avec d'autres pistes. Pour contourner cet obstacle, nous avons ajouté un **Via** (connecteur vertical) permettant de passer du **Métal 1** au **Métal 2**.



Une fois les Vias positionnés aux extrémités à connecter, nous avons sélectionné la couche **Metal 2**  Metal2 dans la palette des calques. À l'aide de l'outil de tracé (*Create Path*), nous avons dessiné une piste en Métal 2 reliant ces deux Vias.



Enfin, nous avons validé notre conception physique par :

1. Une vérification des règles de dessin (**DRC**) pour s'assurer qu'aucune contrainte géométrique du kit gpd180 n'était violée.

```
DRC started.....Thu Dec 11 13:44:24 2025
completed ....Thu Dec 11 13:44:24 2025
CPU TIME = 00:00:00 TOTAL TIME = 00:00:00
***** Summary of rule violations for cell "NAND layout" *****
Total errors found: 0
```

Une comparaison schéma-layout (**LVS**) qui a confirmé la correspondance parfaite entre notre schéma électrique et le layout réalisé.

```
The net-lists match.

                                layout schematic
                                instances
un-matched                      0          0
rewired                         0          0
size errors                     0          0
pruned                          0          0
active                          4          4
total                           4          4

                                nets
un-matched                      0          0
merged                         0          0
pruned                         0          0
active                          6          6
total                           6          6

                                terminals
un-matched                      0          0
matched but
different type                   0          0
total                            5          5
```

6. Conclusion partielle

Cette manipulation nous a permis de concevoir une porte NAND et de valider son fonctionnement logique par simulation. La partie la plus instructive a été le layout : contrairement à l'inverseur, nous avons été confrontés à des croisements de pistes. Nous avons donc appris à utiliser des **Vias** pour passer sur la couche **Métal 2**, une technique indispensable pour router des circuits plus complexes sans créer de courts-circuits. La validation LVS a confirmé que notre conception physique est correcte.

Conclusion Générale

Ce travail pratique nous a permis de parcourir l'intégralité du flux de conception de circuits intégrés numériques sous Cadence Virtuoso. En partant de l'inverseur simple pour aller vers une porte NAND, nous avons acquis les compétences fondamentales en saisie de schéma, création de symboles, simulation analogique (DC et Transitoire) et dessin de layout. La validation finale par DRC et LVS garantit que les circuits conçus sont fonctionnels et prêts pour une éventuelle fabrication.

Annexe

Vérification de NAND layout

